

Roger Finnerty

ME570 A1

Sept 19, 2023

Homework 1 Report

1.1) The following figures show two plotted polygons, one with clockwise-ordered vertices (hollow) and the other with counterclockwise-order vertices (solid).

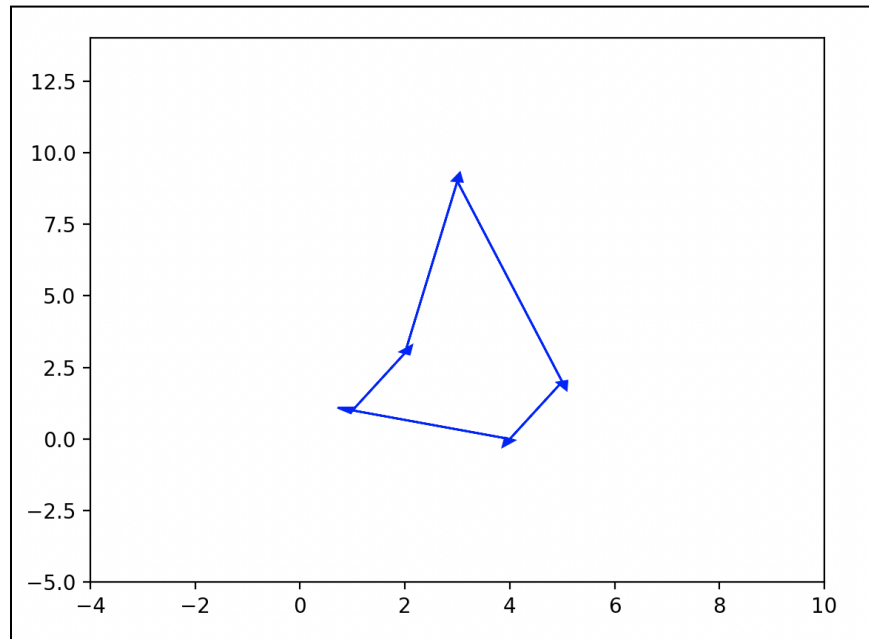


Figure 1a) Clockwise (hollow) polygon

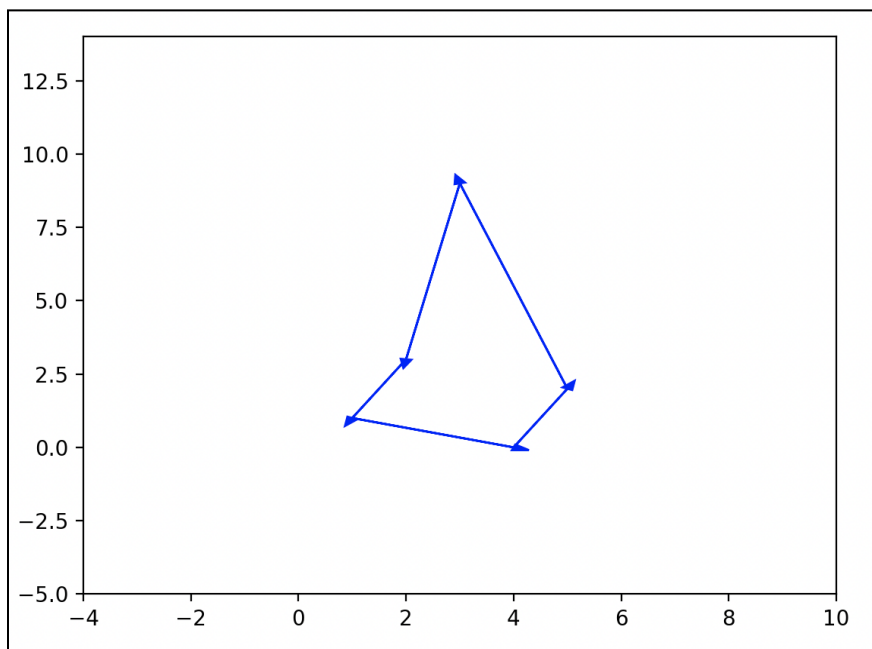


Figure 1b) Counter-clockwise (solid) polygon

1.2) The figures below show the results of five runs of the *is_collision* test for the Edge class which determines if the edge is in collision with a second test line. The red lines are in collision while green are not.

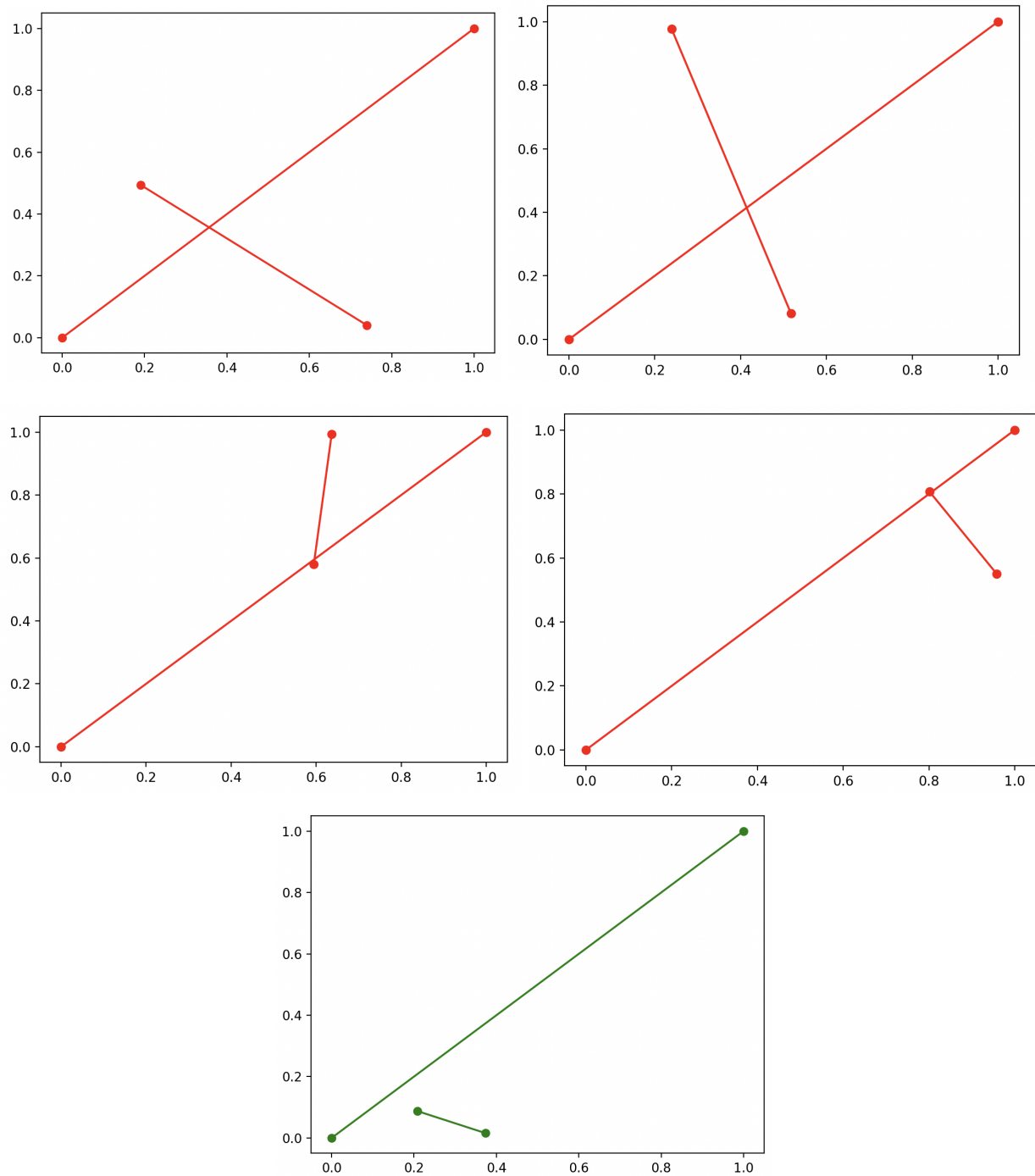
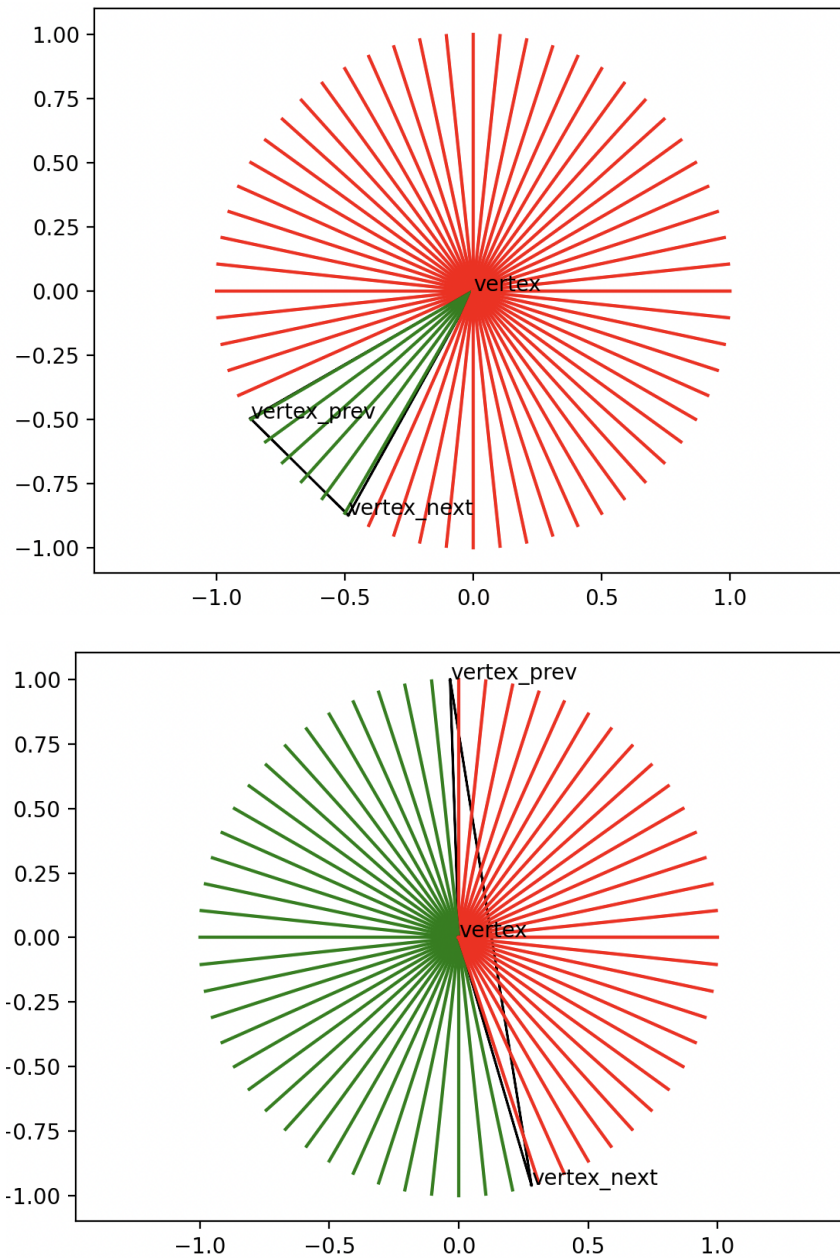


Figure 2) Results of 5 edges collision test runs

1.3)

The variables 's_angle' and 'c_angle' represent the sine and cosine of the angle between the edges. The edge angle is then computed by taking the arctangent of the ratio of the sine and cosine.

1.4) The following figures show verification of the *is_self_occluded* test, which determines whether a test point is self-occluded given the corner of a polygon.



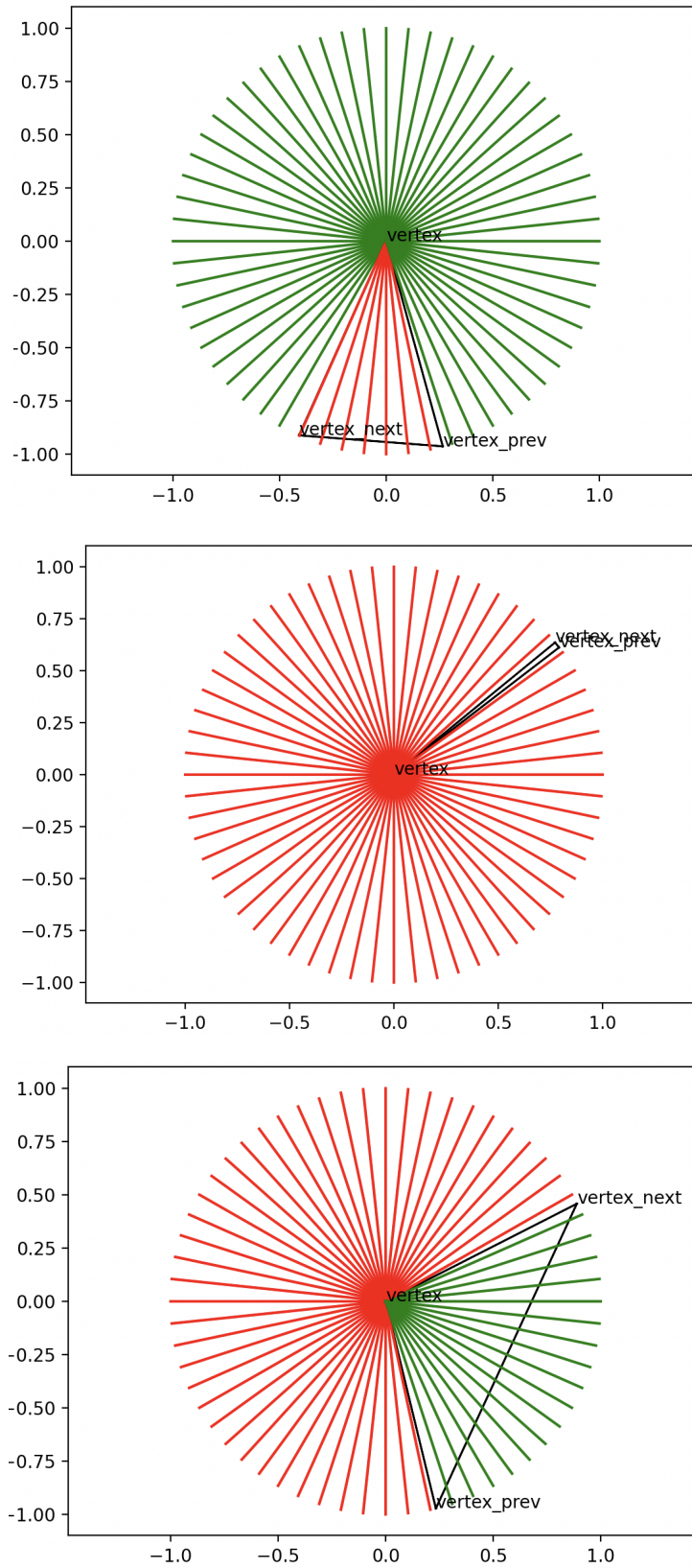
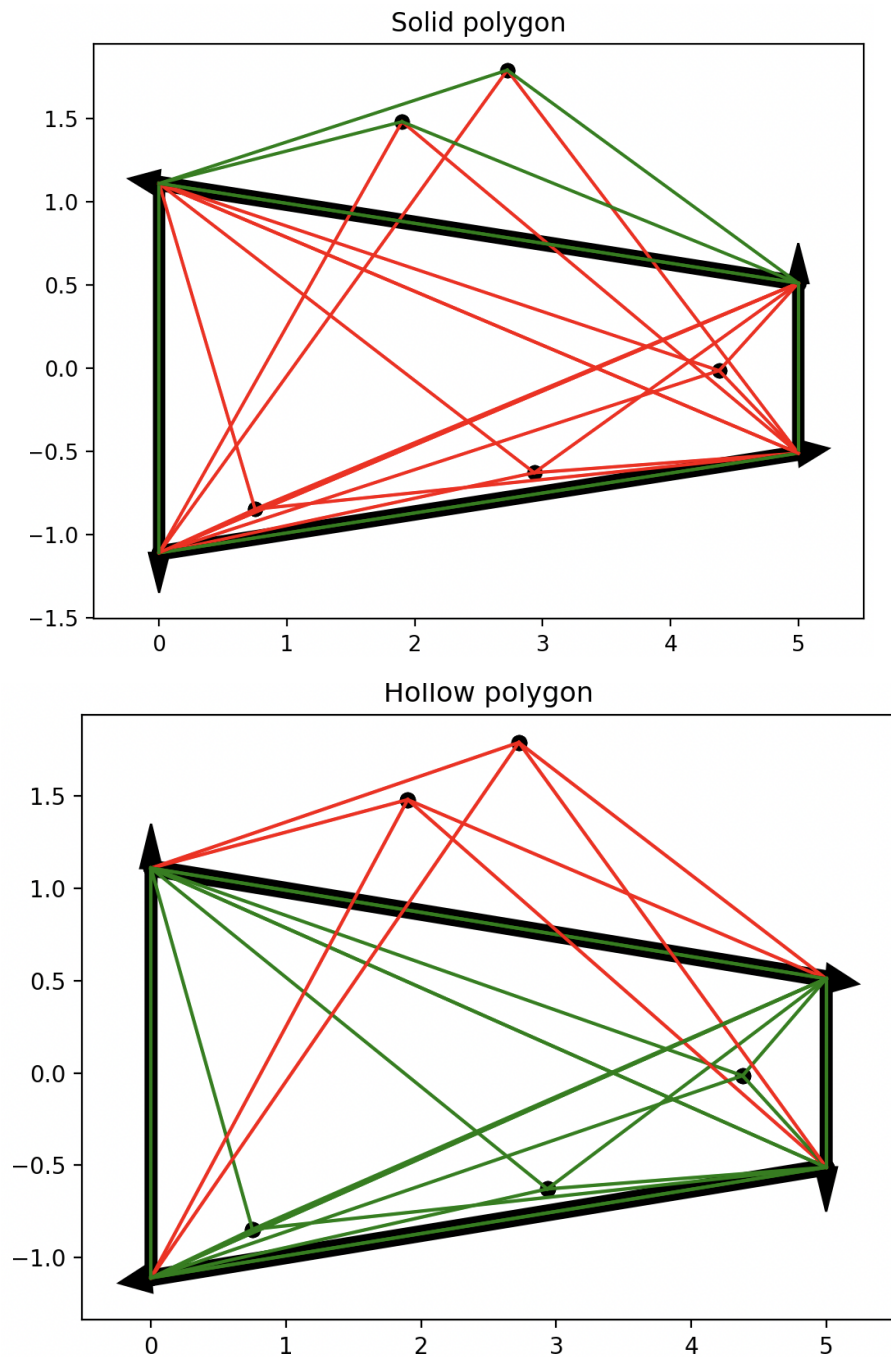


Figure 3) Five instances of *is_self_occluded_test*

1.5) The following figures show four instances of the *is_visible* test with an array of test points, one for each of two different polygons, both hollow and solid. The *is_visible* test determines if a test point is visible from a vertex of a polygon and draws a line connecting the two. If the point is visible, the line is green, if not, the line is red.



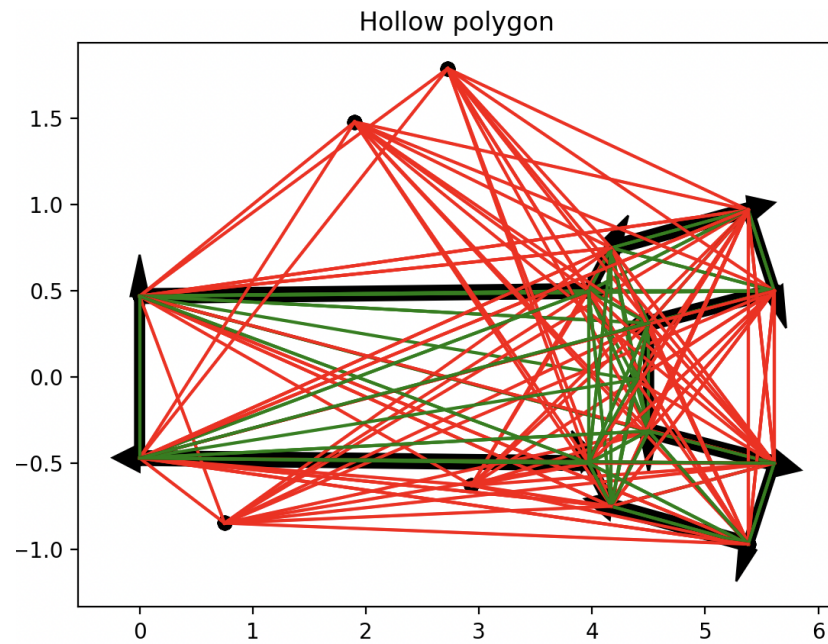
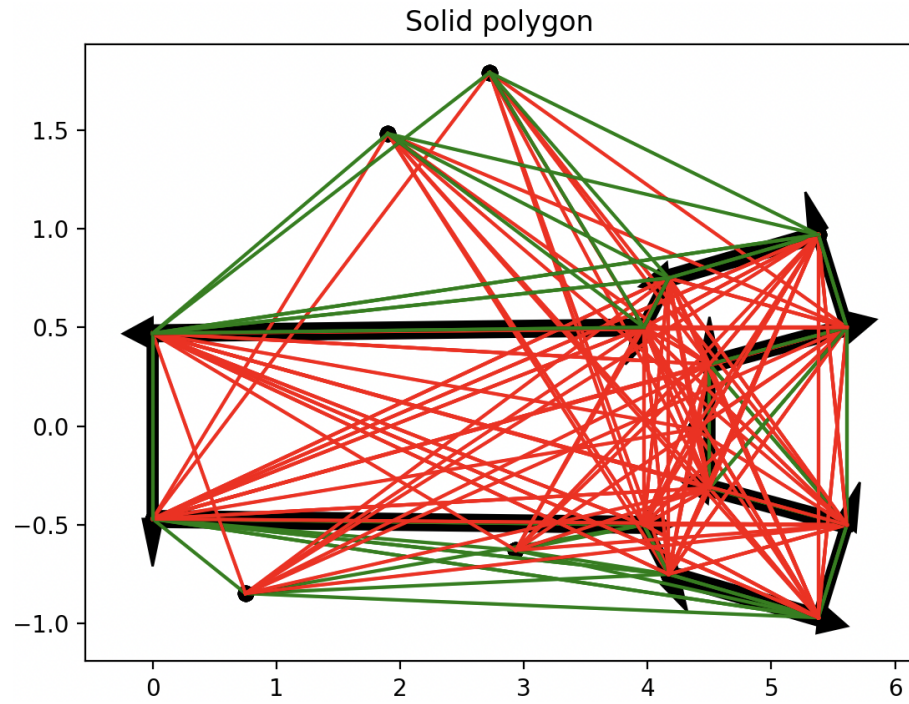
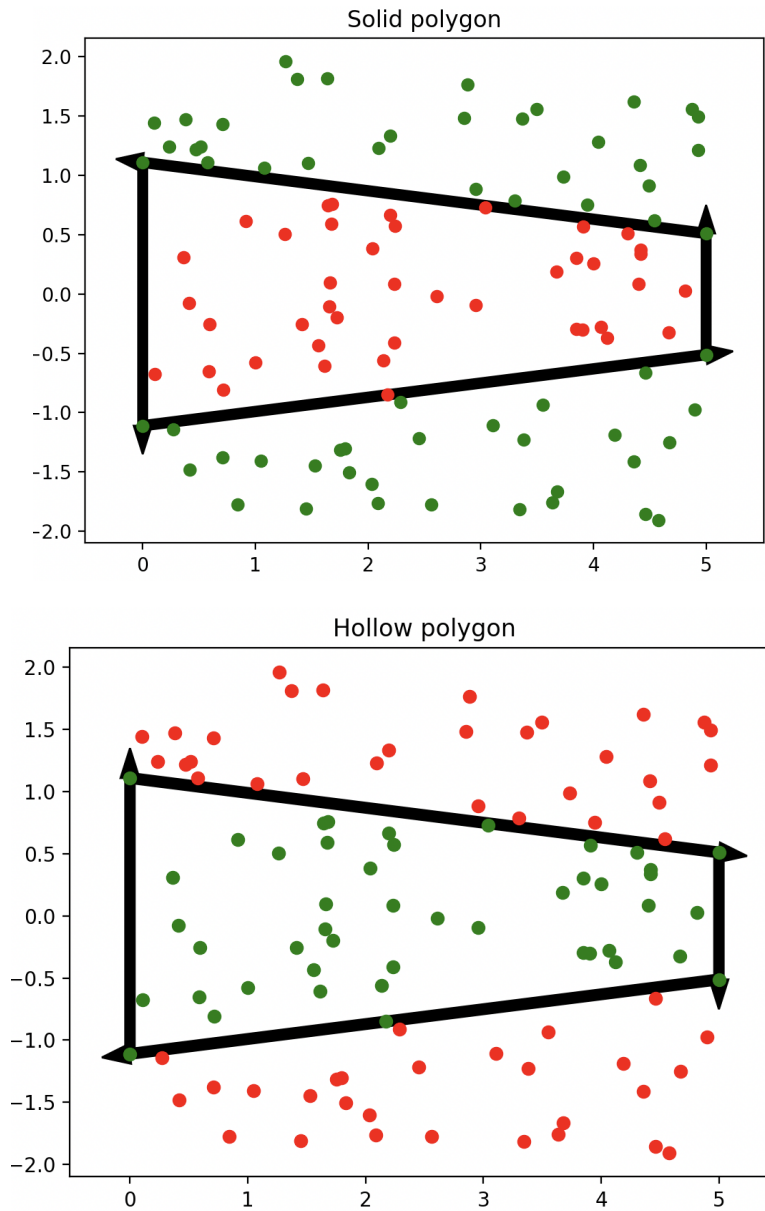


Figure 4) Instances of *polygon_is_visible_test*

1.6)

The figures below show the results of *polygon_is_collision_test*, which determines whether each test point in an array is in collision with the polygon, that is, outside for a hollow polygon and inside for a solid polygon.



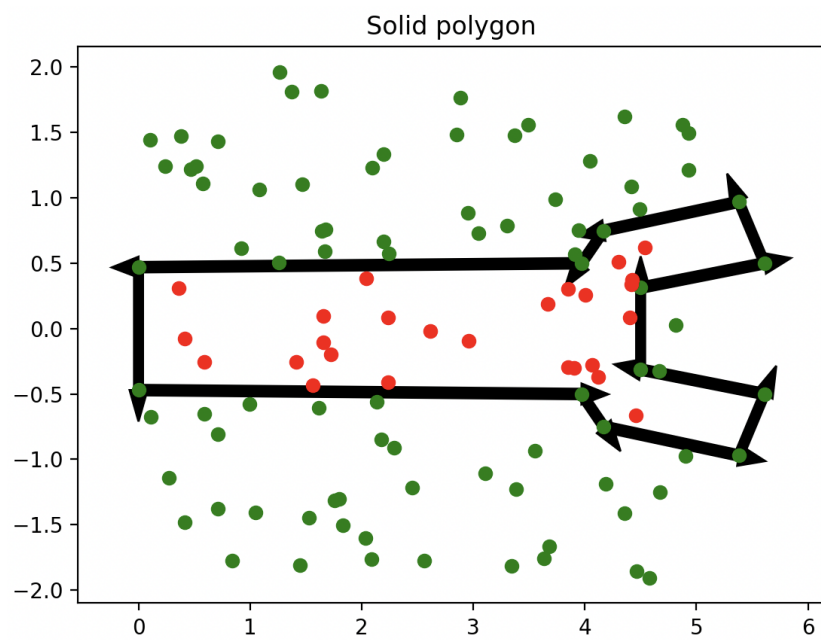
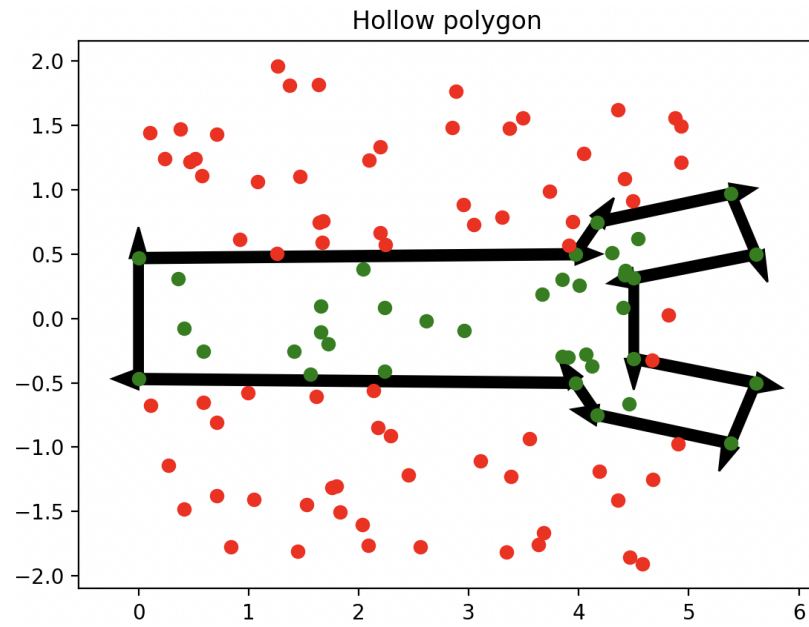


Figure 5) Four instances of *polygon_is_collision_test*

2.1)

The figure below shows the output of the *priority_test* function, which demonstrates the functionality of the priority queue. Beginning with an empty queue, 3 elements are added, then the minimum element is extracted and printed, then another element is added, then certain elements are checked for presence in the queue, then all elements are removed by repeated extractions.

```
● (base) crc-dot1x-nat-10-239-114-26:homework1 rogerfinnerty$ ./me570_hw1.py
Queue is empty.

Current queue:
Key: Oranges, Cost($): 4.5.
Key: Apples, Cost($): 1.
Key: Bananas, Cost($): 2.7.

Minimum key: Apples, cost($): 1
Current queue:
Key: Oranges, Cost($): 4.5.
Key: Apples, Cost($): 1.
Key: Bananas, Cost($): 2.7.
Key: Cantaloupe, Cost($): 3.

Apples is present in the queue.
Bananas is present in the queue.
(1,5) is not present in the queue.

Removed queue item with key: Cantaloupe, value: 3
Current queue:
Key: Oranges, Cost($): 4.5.
Key: Apples, Cost($): 1.
Key: Bananas, Cost($): 2.7.
Removed queue item with key: Bananas, value: 2.7
Current queue:
Key: Oranges, Cost($): 4.5.
Key: Apples, Cost($): 1.
Removed queue item with key: Apples, value: 1
Current queue:
Key: Oranges, Cost($): 4.5.
Removed queue item with key: Oranges, value: 4.5
Queue is empty.
```

Figure 6) Priority queue functionality

2.2)

This function in the PriorityQueue displays the elements of the queue in descending order by cost.

```
def display_descending(self):
    """
    Displays the elements of the queue in descending order by cost.
    """
    # First create a duplicate priority queue so that we can delete elements as they are printed
    queue_copy = PriorityQueue()
    for i in range(len(self.queue_list)):
        key = self.queue_list[i][0]
        cost = self.queue_list[i][1]
        queue_copy.insert(key, cost)

    # Modified version of extract_min function to find the index of the maximum cost item, print that item,
    # then remove it from the list
    for i in range(len(self.queue_list)):
        # Determine index of maximum value
        max_idx = 0
        for j in range(1, len(queue_copy.queue_list)):
            if queue_copy.queue_list[j][1] > queue_copy.queue_list[max_idx][1]:
                max_idx = j
        # Display the maximum key, value pair
        print(f"Key: {queue_copy.queue_list[max_idx][0]}, Cost: {queue_copy.queue_list[max_idx][1]}")
        # Remove the maximum pair from the list
        queue_copy.queue_list.pop(max_idx)
```

The following is a sample output generated from generic coordinates with arbitrary costs associated.

```
● (base) crc-dot1x-nat-10-239-114-26:homework1 rogerfinnerty$ ./me570_hw1.py
Original priority queue:
Current queue:
Key: (x0, y0), Cost: 4.5.
Key: (x1, y1), Cost: 1.
Key: (x2, y2), Cost: 2.7.
Key: (x3, y3), Cost: 3.

Sorted priority queue:
Key: (x0, y0), Cost: 4.5
Key: (x3, y3), Cost: 3
Key: (x2, y2), Cost: 2.7
Key: (x1, y1), Cost: 1
```

3)

Derivation of the prescribed parametric curve and demonstration that x and $\dot{x}$ are perpendicular.

$$x(t) = \begin{bmatrix} \cos(t^2) \\ \sin(t^2) \end{bmatrix} \rightarrow \dot{x} = \begin{bmatrix} -2t \sin(t^2) \\ 2t \cos(t^2) \end{bmatrix}$$

Any two vectors with a dot (inner) product of 0 are perpendicular.

$$x(t) \cdot \dot{x}(t) = \cos(t^2) \cdot (-2t \sin(t^2)) + \sin(t^2) \cdot (2t \cos(t^2))$$

$$= -2t \cos(t^2) \sin(t^2) + 2t \cos(t^2) \sin(t^2)$$

$$= 0 \Rightarrow x(t), \dot{x}(t) \text{ perpendicular}$$